

docs_chain

Змесціва

Ніводзін адсочаны дакумент не можа выйсці з сінхранізацыі — кантэнт-хэшаваны, двухбаковы, атамарны.

Зводка

Docs Chain — універсальны рухавік распаўсюджвання залежнасцей паміж дакументамі і базамі даных, рэалізаваны на Go, з двухбаковай працай. Калі змяняецца любы элемент зарэгістраванага ланцужка — зыходны Markdown-файл, экспарт у фармаце HTML/PDF/DOCX ці база даных SQLite — сістэма выяўляе змену па хэшы месціва і атамарна распаўсюджвае яе па ўсіх звязаных элементах.

Кароткае апісанне

Рухавік на Go, які падтрымлівае сінхранізацыю дакументаў і баз даных. Выкарыстоўваючы інкрэментнае перавылічэнне з кантэнт-хэшаваннем у стылі Salsa на аснове DAG (тапалагічнае сартаванне па Канах, ранняе спыненне, двухбаковыя рэбры сінхранізацыі, атамарнае перайменаванне + каміты транзакцый SQLite), ён перагенеруе экспарты пры змене любога звязанага артэфакта.

Падрабязнае апісанне

Docs Chain — гэта тое, што ствараецца, калі вы ўжо разоў з дзесятак напісалі той самы няўстойлівы скрыпт «перагенераваць PDF, калі зменіцца Markdown». Ён замяняе ўвесь гэты рукапісны «клей» для сінхранізацыі сапраўдным рухавіком. Сістэма мадэлюе дакументы і базы даных праекта як элементы ланцужка і, калі змяняецца любы з іх, распаўсюджвае гэтую змену па ўсіх звязаных элементах ва ўсіх абвешчаных кірунках — перагенеруючы і пераэкспартуючы атамарна, каб ніводзін адсочаны артэфакт

ніколі не выйшаў з сінхранізацыі. Дызайн запазычвае сваю строгасць непасрэдна з інкрэментных сістэм зборкі, а не са скрыптоў: выяўленне змен адбываецца па **хэшы змесціва, а не па mtime**, таму каманда touch нічога не спрабуе, а аднабайтавая рэдактура запуская менавіта тыя перазборкі, якія патрэбны — без ілжывых трывог і прапушчаных змен. Фармальна гэта можна выразіць адным радком: інкрэментнае перавылічэнне з кантэнт-хэшаваннем у стылі Salsa на аснове DAG, з тапалагічным сартаваннем па Канах, раннім спыненнем, якое адсякае нязмененыя паддрэвы, абвешчанымі двухбаковымі рэбрамі сінхранізацыі з аўтарытэтам, атамарным перайменаваннем і камітамі транзакцый SQLite, каб пры краху падчас распаўсюджвання ніколі не засталася напалову запісанага экспарту. Пастаўляецца як падмадуль vasic-digital і выкарыстоўваецца як асноўны кампанент падмадуля HelixConstitution, таму любы праект, які прымае канстытуцыю, атрымлівае Docs Chain з самага пачатку і рэгіструе ўласныя ланцужкі праз кантэкстныя YAML. Рэалізацыя адкрыта паказвае свой статус (згодна з §11.4.6 канстытуцыі): Этапы 1–4 (асноўны DAG + хэшаванне, адаптары/трансфармацыі вузлоў, аркестратар распаўсюджвання з атамарнасцю, канфігурацыйнае кіраванне некалькімі кантэкстамі CLI з камандамі sync/verify/doctor/graph/watch) рэалізаваны і пратэставаны; Этап 4b дадае ўбудаваныя двухбаковыя пераўтваральнікі md-to-sqlite/sqlite-to-md (чысты Go, дрэйф на ўзроўні радкоў, байтастабільны кругавы абмен) і ўбудаваны colorize-html; Этап 5 — поўнамаштабнае рэальнае бінарнае канцавое тэставанне — рэалізаваны і ПРАЙШОЎ. Этапы 6–7 (распаўсюджванне канстытуцыі, інтэграцыя з ATMOSphere) застаюцца ЗАПЛАНАВАНЫМІ і даступнымі толькі праз аператарныя засаўкі. Herald — першы рэальны спажывец ніжэй па патоку, які сінхранізуе корпус з 66 дакументаў у некалькіх фарматах і пацвярджае чысціню.

Чаму мы яго стварылі

Дакументацыя, экспарты і базы даных разыходзяцца адразу ж, як толькі іх пачынаюць падтрымліваць уручную ці з дапамогай няўстойлівых скрыптоў. Docs Chain робіць сінхранізацыю механічнай, дакладнай па хэшы змесціва і атамарнай, каб змена ў любым месцы ланцужка правільна і бяспечна абнаўляла ўсё, што знаходзіцца ніжэй (і вышэй) па патоку.

Змест

Чаму гэта змяняе правілы гульні

Гэта бярэ дакладнасць і гарантыі, якія аўтары кампілятараў і сістэм зборкі лічаць належнай здабычай — графы залежнасцей з кантролем зместу, мінімальнае перавылічэнне, атамныя каміты — і накіроўвае іх на дакументацыю і базы даных, вобласць, якая гістарычна трымалася на сгоп-заданнях і добрых намерах. Сапраўдная двухбаковая сінхранізацыя азначае, што сувязь паміж крыніцай і яе экспартам кантралюецца ў абодвух кірунках, таму такія праблемы, як “дакументацыя састарэла” ці “экспарт не адпавядае крыніцы”, перастаюць быць паўтаральнымі памылкамі і становяцца станамі, якія рухавік проста не дазволіць існаваць.

Што новага

- Інкрэментнае перавылічэнне па графе залежнасцей з кантролем зместу (не па часу змянення) і раннім спыненнем.
- Двухбаковыя сінхранізацыйныя сувязі з дэклараваннем аўтарытэту (дакументацыя ↔ экспарт ↔ SQLite).
- Атамнае перайменаванне + транзакцыйны каміт праз SQLite для бяспечнага распаўсюджвання пры збоях.
- Чысты Go кругавы працэс md-to-sqlite/sqlite-to-md з выяўленнем разыходжанняў на ўзроўні радкоў.

Праблемы і рашэнні

- **Лішнія перазборы:** вырашана дзякуючы кантролю зместу замест штампаў часу.
- **Частковыя/пашкоджаныя абнаўленні:** вырашана з дапамогай атамнага перайменавання і транзакцый SQLite.
- **Правільнае парадкаванне некалькіх удзельнікаў:** вырашана праз тапалагічнае сартаванне Кана з раннім спыненнем.
- **Часнае паведамленне аб магчымасцях:** вырашана праз пазначэнне кожнага этапу як РЭАЛІЗАВАНЫ ці ЗАПЛАНАВАНЫ згодна з §11.4.6.

Тэхнічны стэк (навошта + як)

- **Go** — увесь рухавік (internal/hash, graph, adapter, orchestrator, config, state, runner, cmd/docs_chain).
- **DAG + тапалагічнае сартаванне Кана** — парадкаванне залежнасцей з раннім спыненнем.
- **SQLite (чысты Go modernc)** — удзельнікі базы даных і транзакцыйныя каміты.

- **fsnotify** — дэман watch для жывога распаўсюджвання.
- **YAML config** — рэгістрацыя ланцужкоў па кантэксце.
- **exes: трансфармацыі** — падключальная генерацыя Markdown→HTML/PDF/DOCX.

Адкрытасць дарожнай карты: Этапы 6-7
(распаўсюджванне канстытуцыі, інтэграцыя з
ATMOSphere) ЗАПЛАНАВАНЫ / абмежаваны для
аператара — не ўключаны ў рэліз.